

BRD

Business Requirements Document

PulseAI

Dynamic Customer Journey Risk Prediction Engine

v1.1 — Gradio Edition

Document Type	Business Requirements Document (BRD)
Version	1.1 — Updated: Gradio / Hugging Face Spaces
Status	Draft — For Review
Prepared by	PulseAI Project Team
Date	June 26, 2026

Table of Contents

1. Executive Summary	3
2. Business Context & Problem Statement	3
3. Project Objectives	4
4. Scope	4
5. Stakeholders	5
6. Functional Requirements	5
7. Non-Functional Requirements	7
8. Data Requirements	8
9. System Architecture Overview	8
10. Technology Stack	9
11. Constraints & Assumptions	9
12. Risk Register	10
13. Success Metrics & KPIs	10
14. Project Timeline	11
15. Glossary	11

1 Executive Summary

PulseAI is a machine learning system that monitors each customer's behaviour across every digital touchpoint and predicts high-risk events — such as subscription cancellation (churn), payment default, or policy lapse — before they happen. By surfacing early warning signals, retention teams can act proactively rather than reactively, recovering revenue and improving customer lifetime value.

This document describes the full business requirements for PulseAI — what it must do, why it matters, how it will be built, and how success will be measured. It is intended for technical leads, product managers, and business stakeholders.

v1.1 update: The front-end interface has been updated from Streamlit to Gradio, and the deployment target has changed from Streamlit Cloud to Hugging Face Spaces. All affected sections have been revised accordingly.

Metric	Baseline (Today)	Target (Post-Launch)
Churn prediction accuracy	None (reactive)	AUC-ROC > 0.85
Average time to intervene	After churn occurs	7+ days before event
Manual review effort	100% manual screening	Top 10% flagged automatically
Customer data sources unified	Siloed across 7 systems	Single risk score per customer

Table 1 — Executive KPI Summary

2 Business Context & Problem Statement

2.1 The Problem

Companies in insurance, telecom, banking, healthcare, and e-commerce all face the same challenge: a customer interacts through many disconnected channels — web, mobile app, support calls, email — and by the time a problem is visible (a cancelled policy, a missed payment, an abandoned cart), it is already too late to intervene cost-effectively.

Current approaches rely on monthly reports, spreadsheet rules, or gut instinct. These are slow, inaccurate, and do not adapt to individual customer patterns. There is no system that continuously monitors the full customer journey and assigns a dynamic risk score that updates in real time.

2.2 Industry Examples

Industry	Predicted Event	Current Gap
Insurance	Policy lapse	Detected only after lapse date
Telecom	Subscription churn	Monthly churn report too late
Banking	Loan default	Credit score misses behavioural shift
Healthcare	Medication non-adherence	No automated monitoring system
E-commerce	Cart abandonment	Triggered only at checkout exit

Table 2 — Industry Problem Examples

2.3 Opportunity

Research shows that retaining an existing customer costs 5–7x less than acquiring a new one. A 5% improvement in retention can increase profit by 25–95%. PulseAI directly addresses this opportunity by giving retention teams a real-time, data-driven early-warning system.

3 Project Objectives

Primary Objectives

- Build a real-time customer risk scoring engine using PyTorch sequence models
- Predict churn, default, and lapse events at least 7 days before they occur
- Unify data from PostgreSQL (Supabase) and MongoDB Atlas into a single pipeline
- Expose risk scores via a FastAPI REST endpoint and a Gradio web interface

Secondary Objectives

- Demonstrate sequential event data outperforms static features alone
- Create a reusable ML pipeline applicable across multiple industry verticals
- Build clean, documented code suitable for a production ML portfolio
- Deploy the Gradio app to Hugging Face Spaces for public portfolio visibility

4 Scope

4.1 In Scope

- Data pipeline: pulling and transforming data from Supabase (SQL) and MongoDB Atlas
- Feature engineering from 7 data sources into a model-ready tensor format
- PyTorch LSTM/Transformer model for binary risk classification
- Model training, evaluation (AUC-ROC, F1, precision/recall), and checkpoint saving
- Inference engine: score any customer on demand or in batch
- FastAPI REST endpoints: /score/{id}, /customers, /retrain
- Gradio interface with 4 tabs: Score a Customer, Top Risk Customers, Event Timeline, Retrain Model
- Deployment of the Gradio app to Hugging Face Spaces (free tier)
- Writing results back to the risk_scores table in Supabase
- Full documentation: README, architecture diagram, API docs

4.2 Out of Scope (Phase 1)

- Real-time Kafka/Kinesis event streaming (batch scoring only in Phase 1)
- Mobile application or customer-facing UI
- Multi-tenant support (single company dataset only)
- A/B testing framework for retention interventions
- Integration with CRM systems (Salesforce, HubSpot) — Phase 2
- Causal inference or explainable AI (SHAP/LIME) — Phase 2

5 Stakeholders

Role	Responsibility	Interest in PulseAI
ML Engineer (Owner)	Builds & maintains the model pipeline	Technical quality, model performance
Data Engineer	Manages DB connections and data pipeline	Data reliability, schema consistency
Product Manager	Defines requirements, prioritises features	Business value, on-time delivery
Business Analyst	Validates risk scores with domain expertise	Accuracy, false positive rate
Retention Team	End users of the Gradio interface and API	Ease of use, actionable alerts
Engineering Manager	Reviews architecture, approves deployment	Code quality, scalability

Table 3 — Project Stakeholders

6 Functional Requirements

FR-1 Data Ingestion

ID	Requirement
FR-1.1	Connect to Supabase PostgreSQL and read from all 5 structured tables.
FR-1.2	Connect to MongoDB Atlas and read from all 4 unstructured collections.
FR-1.3	All data must be linkable by customer_id across all 9 sources.
FR-1.4	The pipeline must handle missing events gracefully without crashing.

FR-2 Feature Engineering

ID	Requirement
FR-2.1	Encode customer event sequences as integer token sequences with padding and masking.
FR-2.2	Compute tabular features: payment failure rate, tenure, plan type, ticket count, notification open rate, cancel-page visits.
FR-2.3	Normalise and combine all features into a single input vector per customer.
FR-2.4	Split dataset 70/15/15 into train, validation, and test sets reproducibly.

FR-3 Model Training

ID	Requirement
FR-3.1	Train a PyTorch LSTM model on combined sequence and tabular features.

ID	Requirement
FR-3.2	Log loss, AUC-ROC, F1, precision, and recall at each training epoch.
FR-3.3	Early stopping must halt training if validation loss stagnates for 5 epochs.
FR-3.4	Save the best model checkpoint as a .pt file with metadata.

FR-4 Risk Scoring

ID	Requirement
FR-4.1	Score any customer by ID, returning a risk score (0.0–1.0) and label.
FR-4.2	Write scores to the risk_scores table in Supabase with timestamp and model version.
FR-4.3	Batch scoring must complete within 60 seconds for 10,000 customers.
FR-4.4	Re-score a customer automatically whenever a new event arrives.

FR-5 FastAPI

ID	Requirement
FR-5.1	GET /score/{customer_id} — return latest risk score and label for one customer.
FR-5.2	GET /customers?risk=high — return customers filtered by risk label.
FR-5.3	POST /retrain — trigger a full model retraining cycle.
FR-5.4	All endpoints must return JSON and respond within 500ms for single-customer requests.
FR-5.5	API must be secured with an API key header (X-API-Key).

FR-6 Gradio Interface

ID	Requirement
FR-6.1	Tab 1 — Score a Customer: text input for customer_id; outputs risk score gauge, label badge, and top contributing event types.
FR-6.2	Tab 2 — Top Risk Customers: display a sortable gr.DataFrame of the 20 highest-risk customers with score, plan, and tenure.
FR-6.3	Tab 3 — Event Timeline: select a customer and display their chronological event sequence with risk signals highlighted.
FR-6.4	Tab 4 — Retrain Model: trigger retraining and display live epoch-by-epoch metric logs in a gr.Textbox.
FR-6.5	The Gradio app must be deployable to Hugging Face Spaces with a single push to the main branch.
FR-6.6	The app must expose its auto-generated Gradio API (/api/predict) for programmatic access alongside FastAPI.

7 Non-Functional Requirements

ID	Category	Requirement	Target
NFR-1	Performance	Single-customer API response time	< 500ms
NFR-2	Performance	Batch scoring (10,000 customers)	< 60 seconds
NFR-3	Performance	Gradio interface initial load time	< 4 seconds on HF Spaces
NFR-4	Scalability	API must be stateless and containerisable	Docker-ready
NFR-5	Reliability	API availability target	> 99% uptime
NFR-6	Security	All FastAPI endpoints authenticated	X-API-Key required
NFR-7	Security	No customer PII in application logs	IDs only — no names/emails
NFR-8	Maintainability	Unit test line coverage	> 70%
NFR-9	Portability	Runs in Python virtual environment	Python 3.10+
NFR-10	Observability	Training metrics logged per epoch	CSV log or Weights & Biases
NFR-11	Data quality	Pipeline handles missing MongoDB documents	Graceful fallback to defaults

Table 4 — Non-Functional Requirements

8 Data Requirements

8.1 Data Sources

PulseAI consumes data from two databases. PostgreSQL (hosted on Supabase) holds structured, relational data: customer profiles, transactions, subscriptions, and model outputs. MongoDB Atlas holds unstructured, variable-length data: event logs, support ticket conversations, session page sequences, and notification engagement records.

Source	Type	Tables / Collections	Records
Supabase (PostgreSQL)	Structured SQL	customers, transactions, subscriptions, risk_scores, model_training_labels	100 / 3,847 / 150
MongoDB Atlas	Unstructured NoSQL	customer_events, support_tickets, behavioral_sessions, notification_responses	100 / 317 / 2,655 / 100

Table 5 — Data Sources Summary

8.2 Data Quality Requirements

- All customer_id values must be consistent across all 9 tables/collections.
- Timestamps must be ISO-8601 format with timezone (UTC).

- `payment_status` must be one of: success, failed, pending.
- `risk_score` values must be in range [0.0, 1.0].
- Event sequences must preserve chronological order.
- Missing or null optional fields must not cause pipeline failures.

9 System Architecture Overview

The PulseAI architecture follows a four-layer design: Data Layer, Processing Layer, Model Layer, and Serving Layer. The Serving Layer now comprises both FastAPI (for programmatic/machine access) and Gradio (for human/visual access), deployed to Hugging Face Spaces.

Layer	Components	Responsibility
Data Layer	Supabase PostgreSQL MongoDB Atlas	Store all raw customer data — structured (SQL) and unstructured (NoSQL)
Processing Layer	Python pipeline Feature engineering Tensor builder	Pull from both DBs, compute features, pad sequences, produce .pt tensors
Model Layer	PyTorch LSTM encoder Tabular MLP encoder Fusion classifier	Learn risk patterns from labelled history and output risk probabilities
Serving Layer	FastAPI REST API Gradio Interface (4 tabs) Hugging Face Spaces deployment Supabase write-back	Expose scores to humans via Gradio and to systems via FastAPI. Deployed publicly on HF Spaces.

Table 6 — System Architecture Layers

9.1 Data Flow Summary

1. Raw customer events arrive in MongoDB; transactions and profiles sit in Supabase. 2. The feature pipeline pulls both, joins on customer_id, engineers features, and saves a .pt dataset. 3. The PyTorch training loop trains the model and saves the best checkpoint. 4. The inference engine loads the checkpoint and scores customers. 5. Scores are written back to Supabase risk_scores. 6. FastAPI exposes scores to downstream systems. 7. The Gradio interface provides a visual, tab-based interface for human users — hosted on Hugging Face Spaces.

10 Technology Stack

Category	Technology	Version	Purpose
Language	Python	3.10+	Core programming language
Deep Learning	PyTorch	2.x	Model architecture, training, inference
Data (SQL)	Supabase + psycopg2	Latest	PostgreSQL hosting and Python connector
Data (NoSQL)	MongoDB Atlas + pymongo	Latest	Document store and Python driver
API Framework	FastAPI	0.110+	REST API for programmatic model serving
ML Interface	Gradio	4.x	Tab-based web interface for human users
Deployment	Hugging Face Spaces	Free tier	Public hosting of the Gradio interface
Data Processing	Pandas + NumPy	2.x	Feature engineering and wrangling
Model Tracking	CSV / Weights & Biases	Optional	Training metric logging per epoch
Version Control	Git + GitHub	Latest	Source code and HF Spaces deployment trigger

Table 7 — Technology Stack (v1.1 — Gradio Edition)

11 Constraints & Assumptions

11.1 Constraints

- All infrastructure must use free tiers only: Supabase Free, MongoDB Atlas M0, Hugging Face Spaces Free.
- The model must run on a standard laptop CPU — no GPU required for Phase 1.
- Phase 1 uses 100 synthetic customers — production scaling is Phase 2.
- The project must be completable within a 12-week solo development timeline.
- No paid third-party ML platforms in Phase 1.
- The Gradio app on Hugging Face Spaces may go to sleep after inactivity (free tier limit) — acceptable for Phase 1.

11.2 Assumptions

- The synthetic dataset faithfully represents real customer behaviour patterns.
- Customer IDs are globally unique and consistent across all data sources.
- All timestamps are stored in UTC and accurate to the second.
- The business defines churn as a cancelled subscription or 90+ days of inactivity.
- Re-training will be triggered manually in Phase 1 via the Gradio Retrain tab.
- Network access to Supabase and MongoDB Atlas is available during development.

12 Risk Register

Risk ID	Risk Description	Likelihood	Impact	Mitigation
R-01	Synthetic data does not reflect real patterns — poor model performance	Medium	High	Validate feature distributions; add noise injection
R-02	Supabase or MongoDB free tier has downtime	Low	Medium	Add retry logic; cache last-known data locally
R-03	PyTorch model overfits on small (100 customer) dataset	High	High	Use dropout, L2 regularisation, cross-validation
R-04	Hugging Face Spaces cold-start delay (free tier sleep)	Medium	Low	Acceptable for Phase 1; upgrade to persistent Space in Phase 2
R-05	Feature engineering takes longer than estimated	Medium	Low	Phase 2 tasks are buffer; pipeline is highest priority
R-06	Model accuracy below AUC-ROC 0.85 target	Medium	High	Tune hyperparameters; consider Transformer encoder
R-07	Gradio version incompatibility with HF Spaces runtime	Low	Medium	Pin Gradio version in requirements.txt; test locally first

Table 8 — Risk Register

13 Success Metrics & KPIs

KPI	Definition	Target	How to Measure
Model AUC-ROC	Area under ROC curve on test set	> 0.85	sklearn.metrics on held-out test set
F1 Score	Harmonic mean of precision and recall	> 0.78	sklearn.metrics, threshold = 0.5
Precision (High Risk)	% of flagged customers who actually churned	> 70%	Confusion matrix on test set
FastAPI response time	95th percentile single-score latency	< 500ms	Load test with k6 or locust
Gradio interface load	Time to first render on HF Spaces	< 4s	Manual timing after cold start
Data pipeline runtime	End-to-end feature build time	< 120s for 100 customers	Time the pipeline script
Code coverage	Unit test line coverage	> 70%	pytest --cov report

Table 9 — Success Metrics & KPIs

14 Project Timeline

Phase	Weeks	Key Deliverables
1 — Setup & Infrastructure	1–2	GitHub repo, Supabase connector, MongoDB connector, README
2 — Data Pipeline & Feature Engineering	3–4	Feature matrix, padded sequences, .pt tensors, event vocabulary
3 — Model Architecture & Training	5–7	LSTM model, training loop, AUC-ROC report, best .pt checkpoint
4 — Risk Scoring Engine	8	Inference pipeline, risk labels, Supabase write-back, unit tests
5 — FastAPI + Gradio Interface	9–10	FastAPI endpoints, Gradio 4-tab app, Hugging Face Spaces deployment
6 — Documentation & Portfolio	11–12	Full README, architecture diagram, demo video, LinkedIn post

Table 10 — Project Timeline (v1.1)

15 Glossary

AUC-ROC	Area Under the Receiver Operating Characteristic curve. Measures how well a binary classifier separates classes. Score 1.0 = perfect; 0.5 = random guess.
Churn	A customer cancelling their subscription or becoming inactive for 90+ days.
Customer Journey	The complete, ordered sequence of interactions a customer has with a product over time.
FastAPI	Modern Python web framework used to build the REST API that exposes PulseAI risk scores to programmatic callers.
Feature Engineering	Transforming raw data into numerical inputs suitable for a machine learning model.
Gradio	Open-source Python library for building interactive ML web interfaces. Used for PulseAI's human-facing interface with 4 tabs: Score a Customer, Top Risk Customers, Event Timeline, and Retrain Model.
Hugging Face Spaces	Free cloud hosting platform for Gradio (and Streamlit) ML apps. PulseAI's Gradio interface is deployed here as a public portfolio artifact.
LSTM	Long Short-Term Memory — a PyTorch recurrent neural network designed to learn patterns in sequences over time.
Risk Score	A number between 0.0 and 1.0. Higher = higher probability of a negative event. Labels: 0.0–0.3 Low, 0.3–0.6 Medium, 0.6–0.8 High, 0.8–1.0 Critical.
Supabase	Cloud-hosted PostgreSQL service used as the structured data store for PulseAI.
MongoDB Atlas	Cloud-hosted MongoDB service used as the unstructured data store for PulseAI.
Tensor	A multi-dimensional array used by PyTorch to represent data — the format the model reads as input.

End of Document — PulseAI Business Requirements Document v1.1 (Gradio Edition)